

01 | 快速入门：Rust中有哪些你不得不了解的基础语法？

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天是我们的 Rust 入门与实战第一讲。

无论对人，还是对事儿，第一印象都很重要，Rust 也不例外。今天我们就来看一看 Rust 给人的第一印象是什么吧。其实 Rust 宣称的安全、高性能、无畏并发这些特点，初次接触的时候都是感受不到的。第一次能直观感受到的实际是下面这些东西。

Rust 代码长什么样儿？

Rust 在编辑器里面体验如何？

Rust 工程如何创建？

Rust 程序如何编译、执行？

下面我们马上下载安装 Rust，快速体验一波。

下载安装

要做 Rust 编程开发，安装 Rust 编译器套件是第一步。如果是在 MacOS 或 Linux 下，只需要执行：

```
1 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

 复制代码

按提示执行操作，就安装好了，非常方便。

而如果你使用的是 Windows 系统，那么会有更多选择。你既可以在 WSL 中开发编译 Rust 代码，也可以在 Windows 原生平台上开发 Rust 代码。

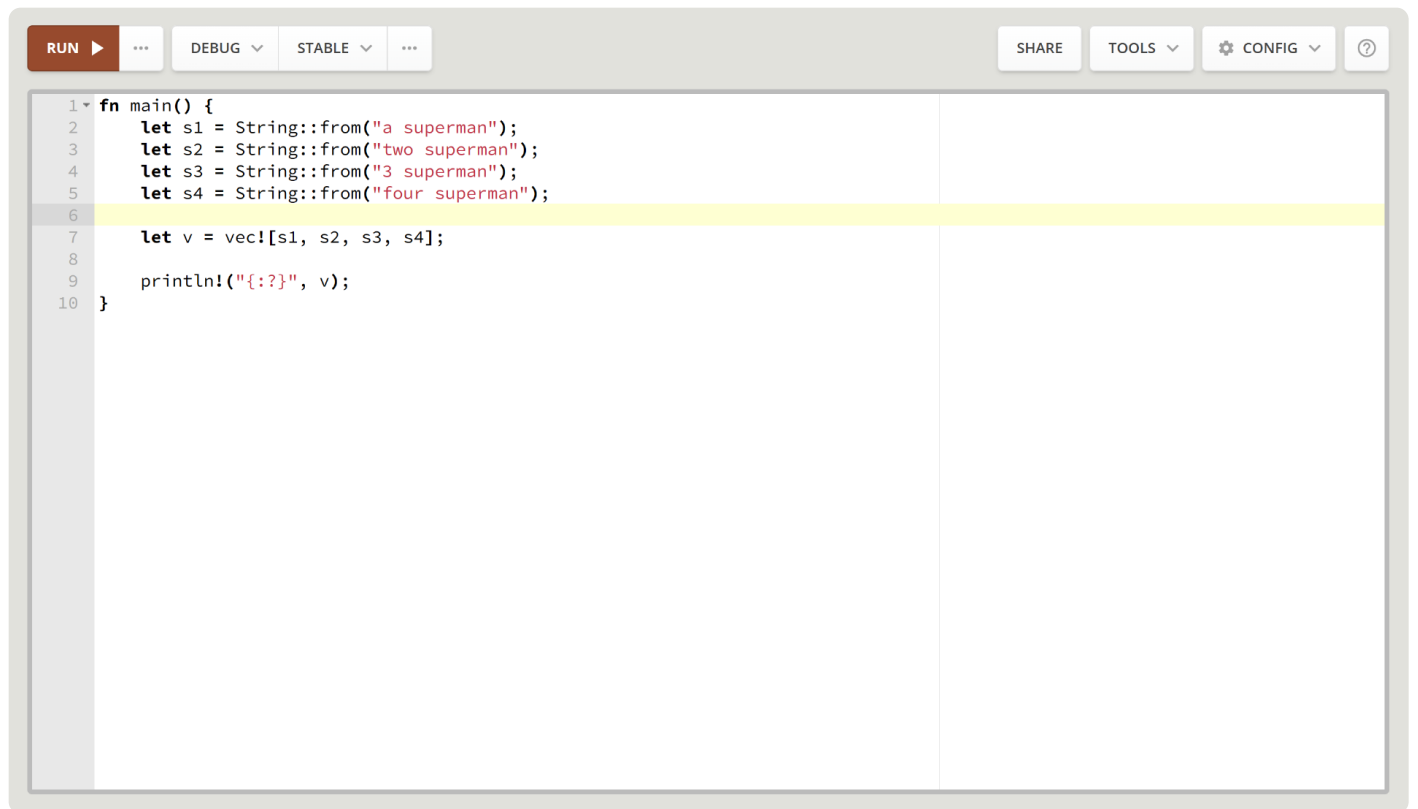
如果你计划在 WSL 中开发，安装方式与上面一致。

```
1 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

 复制代码

如果想在 Windows 原生平台上开发 Rust 代码，首先需要确定安装 [32 位的版本](#) 还是 [64 位的版本](#)。在安装过程中，它会询问你是想安装 GNU 工具链的版本还是 MSVC 工具链的版本。安装 GNU 工具链版本的话，不需要额外安装其他软件包。而安装 MSVC 工具链的话，需要先安装微软的 [Visual Studio](#) 依赖。

如果你暂时不想在本地安装，或者本地安装有问题，对于我们初学者来说，也有一个方便、快捷的方式，就是 Rust 语言官方提供的一个网页端的 [Rust 试验场](#)，可以让你快速体验。

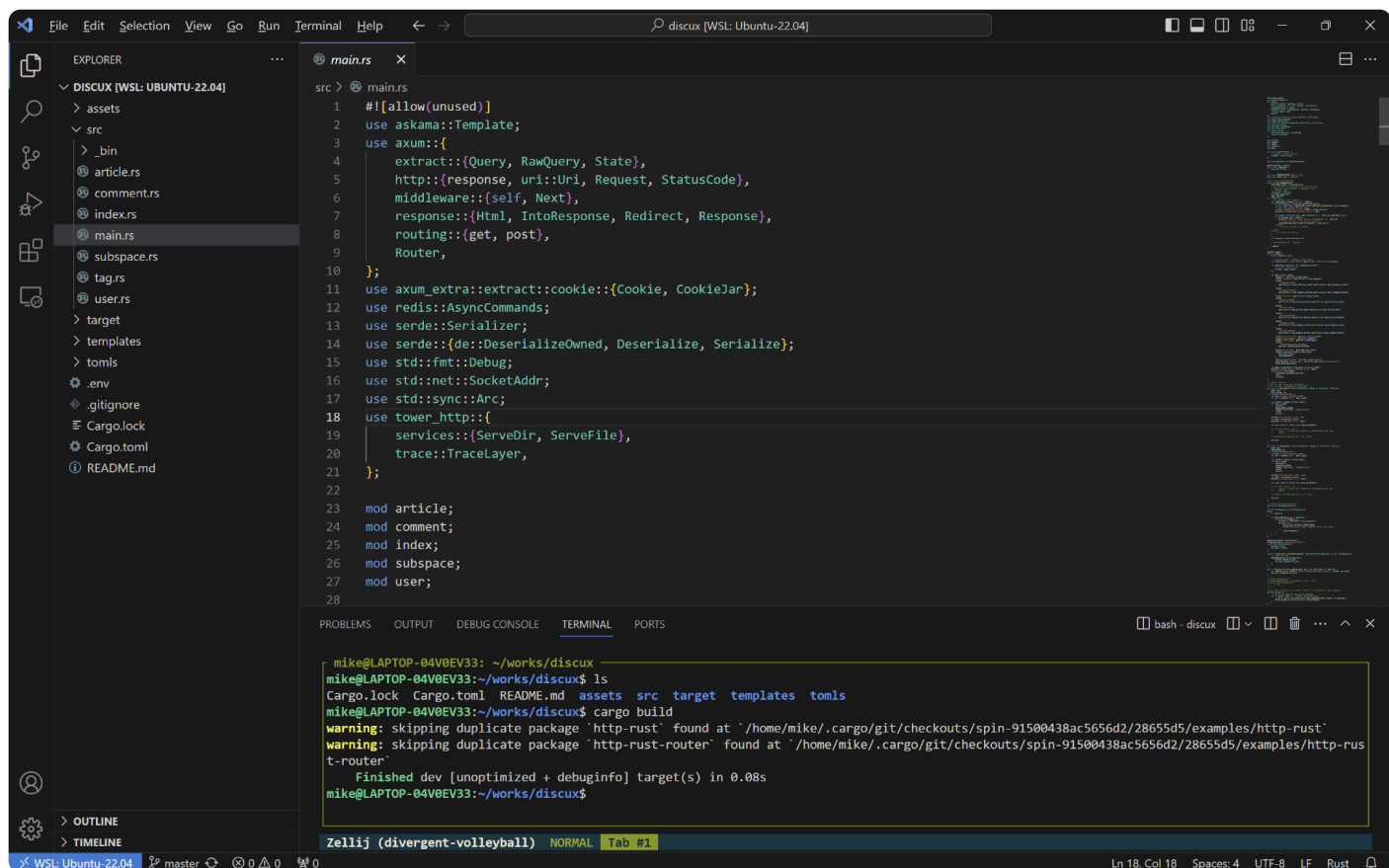


网页端的 Rust 试验场：<https://play.rust-lang.org/>

这个网页 Playground 非常方便，可以用来快速验证一些代码片段，也便于将代码分享给别人。如果你的电脑本地没有安装 Rust 套件，可以临时使用这个 Playground 学习。

编辑器 / IDE

开发 Rust，除了下载、安装 Rust 本身之外，还有一个工具也推荐你使用，就是 **VS Code**。需要提醒你的是，在 VS Code 中需要安装 rust-analyzer 插件才会有自动提示等功能。你可以看一下 VS Code 编辑 Rust 代码的效果。



VS Code 功能非常强大，除了基本的 IDE 功能外，还能实现**远程编辑**。比如在 Windows 下开发，代码放在 WSL Linux 里面，在 Windows Host 下使用 VS Code 远程编辑 WSL 中的代码，体验非常棒。

其他一些常用的 Rust 代码编辑器还有 VIM、NeoVIM、IDEA、Clion 等。JetBrains 最近推出了 Rust 专用的 IDE：RustRover，如果有精力的话，你也可以下载下来体验一下。

Rust 编译器套件安装好之后，会提供一些工具，这里我们选几个主要的简单介绍一下。

命令	用途	举例
cargo	Rust 的包管理器，构建工具和依赖解决器。可以使用 cargo 命令创建、编辑和构建 Rust 项目	cargo new --bin my_project 可以创建一个名为 my_project 的新的 Rust 项目
rustup	用来升级维护 Rust 编译器套件的版本，同时支持维护多个版本，并可用来安装 Rust 组件	rustup update stable 可将 Rust stable 版本升级至最新
rustfmt	可用来对 Rust 代码按配置格式进行自动排版，用来统一 Rust 代码风格	配合 cargo，直接在工程目录下运行 cargo fmt 就可以对整个工程进行排版
rust-clippy	可用来对 Rust 代码进行严谨性检查，指出一些写得不规范的地方	配合 cargo，直接在工程目录下运行 cargo clippy 就可以对整个工程进行检查了



工具齐备了，下面我们马上体验起来，先来创建一个 Rust 工程。

创建一个工程

创建工程我们应该使用哪个工具呢？没错，就是刚刚我们提到的 cargo 命令行工具。我们用它来创建一个 Rust 工程 helloworld。

打开终端，输入：

```
1 cargo new --bin helloworld
```

复制代码

显示：

```
1 Created binary (application) `helloworld` package
```

复制代码

这样就创建好了一个新工程。这个新工程的目录组织结构是这样的：

 复制代码

```
1 helloworld
2   └─ Cargo.toml
3   └─ src
4       └─ main.rs
```

第一层是一个 src 目录和一个 Cargo.toml 配置文件。src 是放置源代码的地方，而 Cargo.toml 是这个工程的配置文件，我们来看一下里面的内容。

 复制代码

```
1 [package]
2 name = "helloworld"
3 version = "0.1.0"
4 edition = "2021"
5
6 # See more keys and their definitions at https://doc.rust-lang.org/cargo/referenc
7
8 [dependencies]
```

Cargo.toml 中包含 package 等基本信息，里面有名字、版本和采用的 Rust 版次。Rust 3 年发行一个版次，目前有 2015、2018 和 2021 版次，最新的是 2021 版次，也是我们这门课使用的版次。可以执行 `rustc -V` 来查看我们课程使用的 Rust 版本。

 复制代码

```
1 rustc 1.69.0 (84c898d65 2023-04-16)
```

好了，一切就绪后，我们可以来看看 src 下的 main.rs 里面的代码。

Hello, World

 复制代码

```
1 fn main() {
2     println!("Hello, world!");
3 }
```

这段代码的意思是，我们要在终端输出这个 `"Hello, world!"` 的字符串。

使用 `cargo build` 来编译。

 复制代码

```
1 $ cargo build
2   Compiling helloworld v0.1.0 (/home/mike/works/classes/helloworld)
3   Finished dev [unoptimized + debuginfo] target(s) in 1.57s
```

使用 `cargo run` 命令可以直接运行程序。

 复制代码

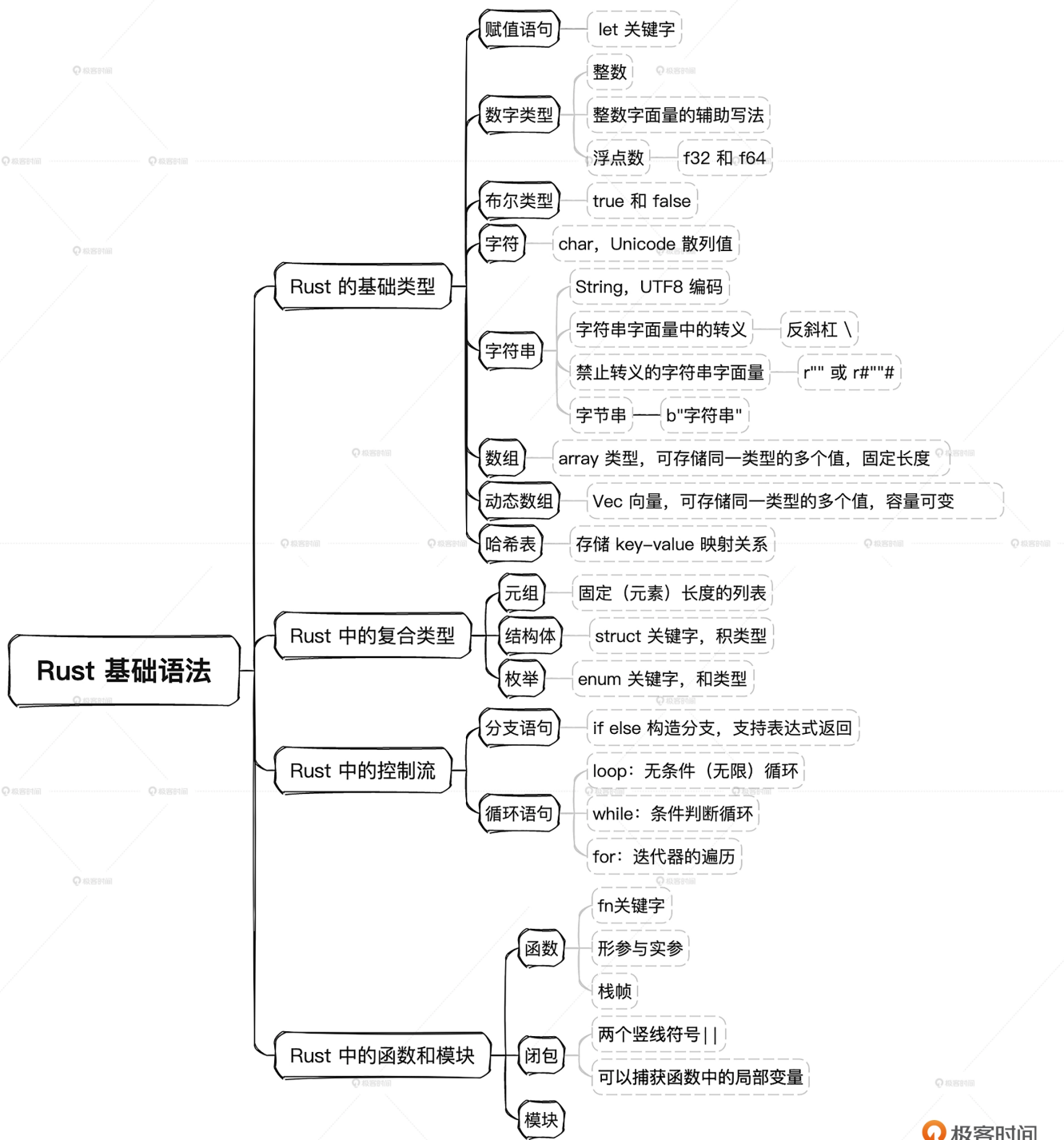
```
1 $ cargo run
2   Finished dev [unoptimized + debuginfo] target(s) in 0.01s
3   Running `target/debug/helloworld`
4 Hello, world!
```

可以看到，最后终端打印出了 `Hello, world`。我们成功地执行了第一个 Rust 程序。

Rust 基础语法

快速体验 Hello World 后，你是不是对 Rust 已经有了一个感性的认识？不过只是会 Hello World 的话，我们离入门 Rust 尚远。下面我们就从 Rust 的基础语法入手开始了解这门语言，为今后使用 Rust 编程打下一个良好的基础。

Rust 基础语法主要包括基础类型、复合类型、控制流、函数与模块几个方面，下面我带你一个一个看。



Rust 的基础类型

赋值语句

Rust 中使用 **let** 关键字定义变量及初始化，你可以看一下我给出的这个例子。

```
1 fn main() {
```

```
2 let a: u32 = 1;  
3 }
```

可以看到，Rust 中类型写在变量名的后面，例子里变量 `a` 的类型是 `u32`，也就是无符号 32 位整数，赋值为 1。Rust 保证你定义的变量在第一次使用之前一定被初始化过。

数字类型

与一些动态语言不同，Rust 中的数字类型是区分位数的。我们先来看整数。

整数

长度	有符号	无符号
8-bit	i8	u8
16-bit	i16	u16
32-bit	i32	u32
64-bit	i64	u64
128-bit	i128	u128
arch	isize	usize



其中，`isize` 和 `usize` 的位数与具体 CPU 架构位数有关。CPU 是 64 位的，它们就是 64 位的，CPU 是 32 位的，它们就是 32 位的。这些整数类型可以在写字面量的时候作为后缀跟在后面，来直接指定值的类型，比如 `let a = 10u32;` 就定义了一个变量 `a`，初始化成无符号 32 位整型，值为 10。

整数字面量的辅助写法

Rust 提供了灵活的数字表示方法，便于我们编写整数字面量。比如：

 复制代码

- 1 十进制字面量 `98_222`，使用下划线按三位数字一组隔开
- 2 十六进制字面量 `0xff`，使用`0x`开头
- 3 8进制字面量 `0o77`，使用`0o`（小写字母o）开头
- 4 二进制字面量 `0b1111_0000`，使用`0b`开头，按4位数字一组隔开
- 5 字符的字节表示 `b'A'`，对一个ASCII字符，在其前面加`b`前缀，直接得到此字符的ASCII码值

各种形式的辅助写法是为了提高程序员写整数字面量的效率，同时更清晰，更不容易犯错。

浮点数

浮点数有两种类型：`f32` 和 `f64`，分别代表 32 位浮点数类型和 64 位浮点数类型。它们也可以跟在字面量的后面，用来指定浮点数值类型，比如 `let a = 10.0f32;` 就定义了一个变量 `a`，初始化成 32 位浮点数类型，值为 10.0。

布尔类型

Rust 中的布尔类型为 `bool`，它只有两个值，`true` 和 `false`。

 复制代码

```
1 let a = true;
2 let b: bool = false;
```

字符

Rust 中的字符类型是 `char`，值用单引号括起来。

 复制代码

```
1 fn main() {
2     let c = 'z';
3     let z: char = 'Z';
4     let heart_eyed_cat = '😻';
5     let t = '中';
6 }
```

Rust 的 `char` 类型存的是 [Unicode 散列值](#)。这意味着它可以表达各种符号，比如中文符号、emoji 符号等。在 Rust 中，`char` 类型在内存中总是占用 [4 个字节](#) 大小。这一点与 C 语言或其他某些语言中的 `char` 有很大不同。

字符串

Rust 中的字符串类型是 `String`。虽然中文表述上，字符串只比前面的字符类型多了一个串字，但它们内部存储结构完全不同。`String` 内部存储的是 Unicode 字符串的 UTF8 编码，而 `char` 直接存的是 Unicode Scalar Value（二者的区别可查阅[这里](#)）。也就是说 **String 不是 char 的数组**，这点与 C 语言也有很大区别。

通过下面示例我们可以看到，Rust 字符串对 Unicode 字符集有着良好的支持。

[复制代码](#)

```
1 let hello = String::from("السلام عليكم");
2 let hello = String::from("Dobrý den");
3 let hello = String::from("Hello");
4 let hello = String::from("ᐃᐣᐣᐣ");
5 let hello = String::from("नमस्ते");
6 let hello = String::from("こんにちは");
7 let hello = String::from("안녕하세요");
8 let hello = String::from("你好");
9 let hello = String::from("Olá");
10 let hello = String::from("Здравствуй");
11 let hello = String::from("Hola");
```

注意，Rust 中的 `String` 不能通过下标去访问。

[复制代码](#)

```
1 let hello = String::from("你好");
2 let a = hello[0]; // 你可能想把“你”字取出来，但实际上这样是错误的
```

为什么呢？你可以想一想。因为 `String` 存储的 Unicode 序列的 UTF8 编码，而 UTF8 编码是变长编码。这样即使能访问成功，也只能取出一个字符的 UTF8 编码的第一个字节，它很可能是没有意义的。因此 Rust 直接对 `String` 禁止了这个索引操作。

字符串字面量中的转义

与 C 语言一样，Rust 中转义符号也是反斜杠 `\`，可用来转义各种字符。你可以运行我给出的这几个示例来理解一下。

[复制代码](#)

```
1 fn main() {
2     // 将""号进行转义
3     let byte_escape = "I'm saying \"Hello\"";
4     println!("{}", byte_escape);
5
6     // 分两行打印
7     let byte_escape = "I'm saying \n 你好";
8     println!("{}", byte_escape);
9
10    // Windows下的换行符
11    let byte_escape = "I'm saying \r\n 你好";
12    println!("{}", byte_escape);
13
14    // 打印出 \ 本身
15    let byte_escape = "I'm saying \\ 0k";
16    println!("{}", byte_escape);
17
18    // 强行在字符串后面加个0，与C语言的字符串一致。
19    let byte_escape = "I'm saying hello.\0";
20    println!("{}", byte_escape);
21 }
```

除此之外，Rust 还支持通过 `\x` 输入等值的 ASCII 字符，以及通过 `\u{}` 输入等值的 Unicode 字符。你可以看一下我给出的这两个例子。

```
1 fn main() {
2     // 使用 \x 输入等值的ASCII字符（最高7位）
3     let byte_escape = "I'm saying hello \xf";
4     println!("{}", byte_escape);
5
6     // 使用 \u{} 输入等值的Unicode字符（最高24位）
7     let byte_escape = "I'm saying hello \u{0065}";
8     println!("{}", byte_escape);
9 }
```

注：字符串转义の詳細知识点，请参考 [Tokens - The Rust Reference \(rust-lang.org\)](https://rust-lang.org/reference/tokens)。

禁止转义的字符串字面量

有时候，我们不希望字符串被转义，也就是想输出原始字面量。这个在 Rust 中也有办法，使用 `r""` 或 `r#"..."#` 把字符串字面量套起来就行了。

```
1 fn main() {
2     // 字符串字面量前面加r，表示不转义
3     let raw_str = r"Escapes don't work here: \x3F \u{211D}";
4     println!("{}", raw_str);
5
6     // 这个字面量必须使用r##这种形式，因为我们在字符串字面量里面保留"
7     let quotes = r#"And then I said: "There is no escape!"#;
8     println!("{}", quotes);
9
10    // 如果遇到字面量里面有#号的情况，可以在r后面，加任意多的前后配对的#号，
11    // 只要能帮助Rust编译器识别就行
12    let longer_delimiter = r####"A string with "#" in it. And even "##!"###;
13    println!("{}", longer_delimiter);
14 }
```

一点小提示，Rust 中的字符串字面量都支持换行写，默认把换行符包含进去。

字节串

很多时候，我们的字符串字面量中用不到 Unicode 字符，只需要 ASCII 字符集。对于这种情况，Rust 还有一种更紧凑的表示法：**字节串**。用 `b` 开头，双引号括起来，比如 `b"this is a byte string"`。这时候字符串的类型已不是字符串，而是字节的数组 `[u8; N]`，`N` 为字节数。

你可以在 Playground 里面运行一下代码，看看输出什么。

[复制代码](#)

```
1 fn main() {
2     // 字节串的类型是字节的数组，而不是字符串了
3     let bytestring: &[u8; 21] = b"this is a byte string";
4     println!("A byte string: {:?}", bytestring);
5
6     // 可以看看下面这串打印出什么
7     let escaped = b"\x52\x75\x73\x74 as bytes";
8     println!("Some escaped bytes: {:?}", escaped);
9
10    // 字节串与原始字面量结合使用
11    let raw_bytestring = br"\u{211D} is not escaped here";
12    println!("{:?}", raw_bytestring);
13 }
```

字节串很有用，特别是在做系统级编程或网络协议开发的时候，经常会用到。

数组

Rust 中的数组是 `array` 类型，用于存储同一类型的多个值。数组表示成 `[T; N]`，由中括号括起来，中间用分号隔开，分号前面表示类型，分号后面表示数组长度。

[复制代码](#)

```
1 fn main() {
2     let a: [i32; 5] = [1, 2, 3, 4, 5];
3     let a = [1, 2, 3, 4, 5];
4 }
```

Rust 中的数组是固定长度的，也就是说在编译阶段就能知道它占用的字节数，并且在运行阶段，不能改变它的长度（尺寸）。

听到这里你是不是想说，这岂不是很麻烦？Rust 中的数组竟然不能改变长度。这里我解释一下，Rust 中区分固定尺寸数组和动态数组。之所以做这种区分是因为 Rust 语言在设计时就要要求适应不同的场合，要有足够的韧性能在不同的场景中都达到最好的性能。因为固定尺寸的数据类型是可以直接放栈上的，创建和回收都比在堆上动态分配的动态数组性能要好。

是否能在编译期计算出某个数据类型在运行过程中占用内存空间的大小，这个指标很重要，Rust 的类型系统就是按这个对类型进行分类的。后面的课程中我们会经常用到这个指标。

数组常用于开辟一个固定大小的 Buffer（缓冲区），用来接收 IO 输入输出等。也常用已知元素个数的字面量集合来初始化，比如表达一年有 12 个月。

[复制代码](#)

```
1 let months = ["January", "February", "March", "April", "May", "June", "July",  
2             "August", "September", "October", "November", "December"];
```

数组的访问，可以用下标索引。

[复制代码](#)

```
1 fn main() {  
2     let a: [i32; 5] = [1, 2, 3, 4, 5];  
3     let b = a[0];  
4     println!("{}", b)  
5 }  
6 // 输出  
7 1
```

我们再来看看，如果下标索引越界了会发生什么。

[复制代码](#)

```
1 fn main() {  
2     let a: [i32; 5] = [1, 2, 3, 4, 5];
```

```
3     let b = a[5];
4     println!("{}", b)
5 }
```

提示：

[复制代码](#)

```
1     Compiling playground v0.0.1 (/playground)
2 error: this operation will panic at runtime
3 --> src/main.rs:3:13
4   |
5 3 |     let b = a[5];
6   |                ^^^^ index out of bounds: the length is 5 but the index is 5
```

这时候你可能已经发现了，Rust 在编译的时候，就给我们指出了问题，说这个操作会在运行的时候崩溃。为什么 Rust 能指出来呢？就是因为**数组的长度是确定的**，Rust 在编译时就分析并提取了这个数组类型占用空间长度的信息，因此直接阻止了你的越界访问。

不得不说，Rust 太贴心了。

动态数组

Rust 中的动态数组类型是 `Vec` (Vector)，也就是向量，中文翻译成动态数组。它用来存储同一类型的多个值，容量可在程序运行的过程中动态地扩大或缩小，因此叫做动态数组。

[复制代码](#)

```
1 fn main() {
2     let v: Vec<i32> = Vec::new();
3     let v = vec![1, 2, 3];
4
5     let mut v = Vec::new();
6     v.push(5);
7     v.push(6);
8     v.push(7);
9     v.push(8);
10 }
```


动态数组可以用下标进行索引访问。

比如：

[复制代码](#)

```
1 fn main() {
2     let s1 = String::from("superman 1");
3     let s2 = String::from("superman 2");
4     let s3 = String::from("superman 3");
5     let s4 = String::from("superman 4");
6
7     let v = vec![s1, s2, s3, s4];
8
9     println!("{:?}", v[0]);
10 }
11 // 输出
12 "superman 1"
```

如果我们下标越界了会发生什么？ Rust 能继续帮我们提前找出问题来吗？ 试一试就知道了。

[复制代码](#)

```
1 fn main() {
2     let s1 = String::from("superman 1");
3     let s2 = String::from("superman 2");
4     let s3 = String::from("superman 3");
5     let s4 = String::from("superman 4");
6
7     let v = vec![s1, s2, s3, s4];
8     // 这里下标访问越界了
9     println!("{:?}", v[4]);
10 }
```

运行后，出现了提示。

[复制代码](#)

```
1 Compiling playground v0.0.1 (/playground)
2 Finished dev [unoptimized + debuginfo] target(s) in 0.62s
3 Running `target/debug/playground`
4 thread 'main' panicked at 'index out of bounds: the len is 4 but the index is 4',
```

5 `note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace`

可以看到，这段代码正确通过了编译，但在运行的时候出错了，并且导致了主线程的崩溃。

你可以将其与前面讲的数组 `array` 下标越界时的预警行为对比理解。为什么 `array` 的越界访问能在编译阶段检查出来，而 `Vec` 的越界访问不能在编译阶段检查出来呢？你可以好好想一想。

哈希表

哈希表是一种常见的结构，用于存储 Key-Value 映射关系，基本在各种语言中都有内置提供。Rust 中的哈希表类型为 `HashMap`。对一个 `HashMap` 结构来说，Key 要求是同一种类型，比如是字符串就统一用字符串，是数字就统一用数字。Value 也是一样，要求是同一种类型。Key 和 Value 的类型不需要相同。

 复制代码

```
1 fn main() {
2     use std::collections::HashMap;
3
4     let mut scores = HashMap::new();
5     scores.insert(String::from("Blue"), 10);
6     scores.insert(String::from("Yellow"), 50);
7 }
```

因为哈希表能从一个键索引到一个值，所以应用场景非常广泛，后面我们还会仔细分析它的使用法。

Rust 中的复合类型

复合类型可以包含多种基础类型，是一种将类型进行有效组织的方式，提供了一级一级搭建更高层类型的能力。Rust 中的复合类型包括元组、结构体、枚举等。

元组

元组是一个固定（元素）长度的列表，每个元素类型可以不一样。用小括号括起来，元素之间用逗号隔开。例如：

[复制代码](#)

```
1 fn main() {  
2     let tup: (i32, f64, u8) = (500, 6.4, 1);  
3 }
```

元组元素的访问：

[复制代码](#)

```
1 fn main() {  
2     let x: (i32, f64, u8) = (500, 6.4, 1);  
3  
4     // 元组使用.运算符访问其元素，下标从0开始，注意语法  
5     let five_hundred = x.0;  
6     let six_point_four = x.1;  
7     let one = x.2;  
8 }
```

与数组的相同点是，它们都是固定元素个数的，在运行时不可伸缩。与数组的不同点是，元组的每个元素的类型可以不一样。元组在 Rust 中很有用，因为它可以用于**函数的返回值**，相当于把多个想返回的值捆绑在一起，一次性返回。

当没有任何元素的时候，元组退化成 `()`，就叫做 `unit` 类型，是 Rust 中一个非常重要的基础类型和值，`unit` 类型唯一的值实例就是 `()`，与其类型本身的表示相同。比如一个函数没有返回值的时候，它实际默认返回的是这个 `unit` 值。

结构体

结构体也是一种复合类型，它由若干字段组成，每个字段的类型可以不一样。Rust 中使用 **struct** 关键字来定义结构体。比如下面的代码就定义了一个 `User` 类型。

[复制代码](#)

```
1 struct User {
```

```
2     active: bool,  
3     username: String,  
4     email: String,  
5     age: u64,  
6 }
```

下面这段代码演示了结构体类型的实例化。

```
1 fn main() {  
2     let user1 = User {  
3         active: true,  
4         username: String::from("someusername123"),  
5         email: String::from("someone@example.com"),  
6         age: 1,  
7     };  
8 }
```

[复制代码](#)

枚举

Rust 中使用 **enum** 关键字定义枚举类型。比如：

```
1 enum IpAddrKind {  
2     V4,  
3     V6,  
4 }
```

[复制代码](#)

枚举类型里面的选项叫做此枚举的变体（variants）。变体是其所属枚举类型的一部分。

枚举使用变体进行枚举类型的实例化，比如：

```
1 let four = IpAddrKind::V4;  
2 let six = IpAddrKind::V6;
```

[复制代码](#)

可以看到，枚举类型也是一种复合类型。但是与结构体不同，结构体类型是里面的所有字段（所有类型）同时起作用，来产生一个具体的实例，而枚举类型是其中的一个变体起作用，来产生一个具体实例，这点区别可以细细品味。学术上，通常把枚举叫作**和类型**（sum type），把结构体叫作**积类型**（product type）。

枚举类型是 Rust 中最强大的复合类型，在后面的课程中我们会看到，枚举就像一个载体，可以携带任何类型。

Rust 中的控制流

下面我们来了解一下 Rust 语言的控制流语句。

分支语句

Rust 中使用 if else 来构造分支。

 复制代码

```
1 fn main() {
2     let number = 6;
3     // 判断数字number能被4, 3, 2中的哪一个数字整除
4     if number % 4 == 0 {
5         println!("number is divisible by 4");
6     } else if number % 3 == 0 {
7         println!("number is divisible by 3");
8     } else if number % 2 == 0 {
9         println!("number is divisible by 2");
10    } else {
11        println!("number is not divisible by 4, 3, or 2");
12    }
13 }
```

与其他 C 系语言不同，Rust 中 if 后面的条件表达式不推荐用 () 包裹起来，因为 Rust 设计者认为那个是不必要的，是多余的语法噪音。

还要注意一点，if else 支持表达式返回。

```
1 fn main() {
2     let x = 1;
3     // 在这里, if else 返回了值
4     let y = if x == 0 {
5         // 代码块结尾最后一句不加分号, 表示把值返回回去
6         100
7     } else {
8         // 代码块结尾最后一句不加分号, 表示把值返回回去
9         101
10    };
11    println!("y is {}", y);
12 }
```

像上面这样的代码，其实已经实现了类似于 C 语言中的三目运算符这样的设计，在 Rust 中，不需要额外提供那样的特殊语法。

循环语句

Rust 中有三种循环语句，分别是 loop、while、for。

loop 用于无条件（无限）循环。

```
1 fn main() {
2     let mut counter = 0;
3
4     // 这里, 接收从循环体中返回的值, 对result进行初始化
5     let result = loop {
6         counter += 1;
7         if counter == 10 {
8             // 使用break跳出循环, 同时带一个返回值回去
9             break counter * 2;
10        }
11    };
12
13    println!("The result is {result}");
14 }
```

请仔细品味这个例子，这种**返回一个值到外面对一个变量初始化的方式**，是 Rust 中的习惯用法，这能让代码更紧凑。

while 循环为条件判断循环。当后面的条件为真的时候，执行循环体里面的代码。和前面的 if 语句一样，Rust 中的 while 后面的条件表达式不推荐用 () 包裹起来。比如：

 复制代码

```
1 fn main() {
2     let mut number = 3;
3
4     while number != 0 {
5         println!("{number}!");
6
7         number -= 1;
8     }
9
10    println!("LIFTOFF!!!");
11 }
```

for 循环在 Rust 中，基本上只用于迭代器（暂时可以想象成对数组，动态数组等）的遍历。Rust 中没有 C 语言那种 for 循环风格的语法支持，因为那被认为是一种不好的设计。

你可以看一下下面的代码，就是对一个数组进行遍历。

 复制代码

```
1 fn main() {
2     let a = [10, 20, 30, 40, 50];
3
4     for element in a {
5         println!("the value is: {element}");
6     }
7 }
```

上面代码对动态数组 Vec 的遍历阻止了越界的可能性，因此用这种方式访问 Vec 比用下标索引的方式访问更加安全。

对于循环的场景，Rust 还提供了一个便捷的语法来生成遍历区间： `..`（两个点）。

请看下面的示例。

```
1 fn main() {
2     // 左闭右开区间
3     for number in 1..4 {
4         println!("{number}");
5     }
6     println!("--");
7     // 左闭右闭区间
8     for number in 1..=4 {
9         println!("{number}");
10    }
11    println!("--");
12    // 反向
13    for number in (1..4).rev() {
14        println!("{number}");
15    }
16 }
17 // 输出
18 1
19 2
20 3
21 --
22 1
23 2
24 3
25 4
26 --
27 3
28 2
29 1
```

我们再来试试打印字符。

```
1 fn main() {
2     for ch in 'a'..'z' {
3         println!("{ch}");
4     }
5 }
6 // 输出:
7 a
8 b
9 c
10 d
11 e
```



```
12 f
13 g
14 h
15 i
16 j
17 k
18 l
19 m
20 n
21 o
22 p
23 q
24 r
25 s
26 t
27 u
28 v
29 w
30 x
31 y
32 z
```

嘿，Rust 很智能啊！

Rust 中的函数和模块

最后我们来看 Rust 的函数、闭包和模块，它们用于封装和复用代码。

函数

函数基本上是所有编程语言的标配，在 Rust 中也不例外，它是一种基本的代码复用方法。在 Rust 中使用 **fn 关键字**来定义一个函数。比如：

[📄 复制代码](#)

```
1 fn print_a_b(a: i32, b: char) {
2     println!("The value of a b is: {a}{b}");
3 }
4
5 fn main() {
6     print_a_b(5, 'h');
7 }
```

函数定义时的参数叫作**形式参数**（形参），函数调用时传入的参数值叫做**实际参数**（实参）。函数的调用要与函数的签名（函数名、参数个数、参数类型、参数顺序、返回类型）一致，也就是实参和形参要匹配。

函数对于几乎所有语言都非常重要，实际上各种编程语言在实现时，都是以函数作为基本单元来组织栈上的内存分配和回收的，这个基本的内存单元就是所谓的**栈帧（frame）**，我们在下节课会讲到。

闭包（Closure）

闭包是另一种风格的函数。它使用两个竖线符号 `||` 定义，而不是用 `fn ()` 来定义。你可以看下面的形式对比。

[复制代码](#)

```
1 // 标准的函数定义
2 fn add_one_v1 (x: u32) -> u32 { x + 1 }
3 // 闭包的定义，请注意形式对比
4 let add_one_v2 = |x: u32| -> u32 { x + 1 };
5 // 闭包的定义2，省略了类型标注
6 let add_one_v3 = |x| { x + 1 };
7 // 闭包的定义3，花括号也省略了
8 let add_one_v4 = |x| x + 1 ;
```

注：可参考完整代码 [🔗 链接](#)

闭包与函数的一个显著不同就是，闭包可以捕获函数中的局部变量为我所用，而函数不行。比如，下面示例中的闭包 `add_v2` 捕获了 `main` 函数中的局部变量 `a` 来使用，但是函数 `add_v1` 就没有这个能力。

[复制代码](#)

```
1 fn main() {
2     let a = 10u32; // 局部变量
3
4     fn add_v1 (x: u32) -> u32 { x + a } // 定义一个内部函数
5     let add_v2 = |x: u32| -> u32 { x + a }; // 定义一个闭包
6
7     let result1 = add_v1(20); // 调用函数
```

```

8     let result2 = add_v2(20); // 调用闭包
9     println!("{}", result2);
10 }

```

这样会编译出错，并提示错误。

[复制代码](#)

```

1 error[E0434]: can't capture dynamic environment in a fn item
2 --> src/main.rs:4:40
3 |
4 4 |         fn add_v1    (x: u32) -> u32 { x + a }    // 定义一个内部函数
5 |                                     ^
6 |
7   = help: use the `|| { ... }` closure form instead

```

闭包之所以能够省略类型参数等信息，主要是其定义在某个函数体内部，从闭包的内容和上下文环境中能够分析出来那些类型信息。

模块

我们不可能把所有代码都写在一个文件里面。代码量多了后，分成不同的文件模块书写是非常自然的事情。这个需求需要从编程语言层级去做一定的支持才行，Rust 也提供了相应的方案。

分文件和目录组织代码理解起来其实很简单，主要的知识点在于目录的组织结构上。比如下面示例：

[复制代码](#)

```

1 backyard
2 |— Cargo.lock
3 |— Cargo.toml
4 |— src
5 |   |— garden           // 子目录
6 |   |   |— vegetables.rs
7 |   |— garden.rs       // 与子目录同名的.rs文件，表示这个模块的入口
8 |— main.rs

```

第五行代码到第七行代码组成 garden 模块，在 garden.rs 中，使用 `mod vegetables;` 导入 vegetables 子模块。

在 main.rs 中，用同样的方式导入 garden 模块。

```
1 mod garden;
```

复制代码

整个代码结构就这样一层一层地组织起来了。

另一种文件的组织形式来自 2015 版，也很常见，有很多人喜欢用。

```
1 backyard
2 |— Cargo.lock
3 |— Cargo.toml
4 |— src
5     |— garden          // 子目录
6         |— mod.rs      // 子目录中有一个固定文件名 mod.rs，表示这个模块的入口
7         |— vegetables.rs
8     |— main.rs
```

复制代码

同上，由第五行到第七行代码组成 garden 模块，在 main.rs 中导入它使用。

你可以在本地创建文件，来体会两种不同目录组织形式的区别。

测试

Rust 语言中自带单元测试和集成测试方案。我们来看一个示例，在 src/lib.rs 或 src/main.rs 下有一段代码。

```
1 fn foo() -> u32 { 10u32 }
2
3 #[cfg(test)]          // 这里配置测试模块
```

复制代码

```

4 mod tests {
5     use crate::foo;
6
7     #[test]                // 具体的单元测试用例
8     fn it_works() {
9         let result = foo();    // 调用被测试的函数或功能
10        assert_eq!(result, 10u32); // 断言
11    }
12 }

```

在项目目录下运行 `cargo test`，会输出类似如下结果。

 复制代码

```

1 running 1 test
2 test tests::it_works ... ok
3 test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finis
4
5 running 0 tests
6 test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finis

```

Rust 自带测试方案，让我们能够非常方便地写测试用例，并且统一了社区的测试设计规范。

配置文件 Cargo.toml

这节课开头的时候，我们简单介绍过 Cargo.toml，它是 Rust 语言包和依赖管理器 Cargo 的配置文件，由官方定义约定。写 Rust 代码基本都会按照这种约定来使用它，对所在工程进行配置。这里面其实包含的知识点非常多，后面实战的部分，我们会详细解释用到的特性。

我们要对包依赖这件事情的复杂度有所了解。不知道你有没有听说过 **npm 依赖黑洞**？指的就是 Node.js 的包依赖太多太琐碎了。这怪不得 Node.js，其实 Rust 也类似。包依赖这事儿，本身就很复杂，可以说这是软件工程固有的复杂性。对固有复杂性，不能绕弯过，只能正面刚。

幸运的是，Cargo 工具已经帮我们搞定了包依赖相关方方面面的麻烦事（此刻 C++ 社区羡慕不已）。为了应对这种复杂性，Cargo 工具提供了非常多的特性，配置起来也相对比较复杂。有兴趣的话，你可以详细了解一下 [🔗 各种配置属性](#)。

小结

这节课我们洋洋洒洒罗列了写一个 Rust 程序所需要用到的基本的语法结构和数据类型，让你对 Rust 语言有了一个初步印象。这些知识点虽多，但并不复杂。因为这节课呈现的绝大部分元素都能在其他语言中找到，所以理解起来应该不算太难。

这节课出现了一个比较重要的指标：**是否能在编译期计算出某个数据类型在运行过程中占用的内存空间的大小**。如果能计算出，那我们称之为固定尺寸的数据类型；如果不能计算出，那我们称之为不固定尺寸的数据类型，或动态数据类型。

其实这也很好理解，因为 Rust 要尽量在编译期间多做一些事情，帮我们做安全性的检查。而在编译期只有能计算出内存尺寸的数据类型，才能被更详尽地去分析和检查，就是这个原理。

思考题

1. Rust 中能否实现类似 JS 中的 number 这种通用的数字类型呢？
2. Rust 中能否实现 Python 中那种无限大小的数字类型呢？

希望你可以积极思考这几个问题，然后把你的答案分享到评论区，如果你觉得这节课对你有帮助的话，也欢迎你分享给你的朋友，邀他一起学习，我们共同进步。下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (33)

最新 精选



学会用实力去谈条件

2023-10-31 来自广西

推荐几个Rust的VsCode插件

Rust Syntax：语法高亮

crates：分析项目依赖

Even Better Toml：项目配置管理

Rust Test Lens：Rust快速测试

作者回复：👍

共 2 条评论 >

👍 10



warning

2023-10-28 来自上海

看完了Rust语言圣经，再看这个还感觉有些吃力需要时不时的看看之前的笔记。课程并不适合没接触过Rust的小白，需要自己花些时间去找别的资料进行补充

作者回复：建议看完这个再去看那个书。因为有些先入为主的理解如果不是很准确的话，会影响后面的学习。

共 5 条评论 >

👍 4



eriklee

2023-10-24 来自北京

IDE推荐一波RustRover，jetbrains家新出的，现在免费使用阶段

作者回复：对的，Rust专用IDE

共 2 条评论 >

👍 3



阿五

2023-10-24 来自广东

虽然 Rust 不像 JavaScript 那样具有动态的通用数字类型，但你可以通过使用 num crate 和 Rust 的强类型系统来实现类似的通用数字操作。

作者回复：厉害的！

共 2 条评论 >

👍 2



知夫

2023-11-11 来自北京

可以推荐一个项目结构的最佳实践吗？能囊括lib，bin，示例，单元测试，性能测试等。

作者回复: <https://github.com/tokio-rs/mini-redis/tree/master> 算比较适合初学者的项目。

共 2 条评论 >

👍 1



J²

2023-11-07 来自广东

工作之余，正在艰难抽空学习rust，希望这次能坚持学完并且入门。

作者回复: 加油



👍 1



🐼

2023-10-24 来自广东

关于char的unicode散列值，gpt4的说明是：

Unicode 是一种字符集 (Character Set)，用于对世界上大多数语言的字符进行编码。不同于 ASCII 码仅包括了基本的英文字符和控制字符，Unicode 意在包括世界上所有的字符，包括字母、符号、表情符号 (emoji) 等。

在 Unicode 标准中，每个字符都有一个唯一的标识符，通常称为代码点 (Code Point)。这些代码点是用整数表示的，一般用十六进制的形式来展示。例如，英文字母 "A" 的 Unicode 代码点是 U+0041，而中文字符 "中" 的 Unicode 代码点是 U+4E2D。

散列值一般用于描述数据结构中用于快速查找的数值标识，但在这里 (Rust 的 char 类型存的是 Unicode "散列值")，这种说法不太准确。实际上，Rust 的 char 类型存储的是 Unicode 代码点。

老师可以解答一下吗

作者回复: 可以看这里: <https://doc.rust-lang.org/std/primitive.char.html#validity> 这里确实比较细节，我是按std标准库文档里面的说法来的。可以参照理解。

共 2 条评论 >

👍 1



uyplayer

2023-10-23 来自上海

```
println!("Hello World! this is first commit in Rust")
```


作者回复: 没有用main()函数包起来呢，怎么编译通过的？

共 2 条评论 >

👍 1



月色很美

2023-12-26 来自浙江

请教下老师，模块管理建议用新的还是2015的呢

作者回复: 都可以，现在社区里面两种势均力敌



HdUIprince

2023-12-26 来自中国香港

"Rust 中的字符串字面量都支持换行写，默认把换行符包含进去。"，老师这句话啥意思呀？

作者回复: 就是 "aaa

Bbb

Ccc"

这样是可以的

